MINISTERIO DE EDUCACION Y CIENCIAS

COLEGIO NACIONAL E.M.D ASUNCIÓN ESCALADA

PROYECTO EDUCATIVO DEL SEGUNDO CURSO
"MODULO RASTREADOR GPS PARA MASCOTAS"

Autores:
Peralta Peralta, Giovanni Luis
Rios Ruiz Diaz, Daiana Beatriz
Santacruz Cubilla, Marcos Damian
Spina Mena, Jose Manuel
Sugastti Caceres, Marina Monserrath

Tutores:
Samuel Bardon

Énfasis:
Bachiller Técnico en Informática

Asunción-Paraguay

2026

Indice

1. Introducción....4
2. Planteamiento del problema....5
2.1 Problemática....5
2.2 Preguntas de investigación....5
3. Justificación....5
4. Objetivos....6
4.1 Objetivo general....6
4.2 Objetivos específicos....6
5. Marco teórico....7
5.1 Antecedentes....7
5.2 Bases teóricas....7
5.2.1 Sistema de posicionamiento global (GPS)....7
5.2.2 Módulo GPS NEO-6M....8
5.2.3 Microcontrolador ESP32....8
5.2.4 Comunicación serial UART....8
5.2.5 Mensajes NMEA y la librería TinyGPSPlus....8
5.2.6 Protocolo HTTP y método POST....8
5.2.7 Node.js, Express y Render....8
5.2.8 Alimentación eléctrica con pilas....9
5.2.9 Soldadura y diseño 3D....9
5.3 Definición de términos....9
6. Marco metodológico....9
6.1 Tipo de investigación....9
6.2 Diseño técnico....10
6.3 Materiales y métodos....10
6.3.2 Herramientas y software....11
6.3.3 Métodos de trabajo....11
7. Desarrollo del proyecto....13
7.1. Diseño del hardware....13
7.1.1. Componentes principales....13
7.1.2. Conexión del módulo GPS con el ESP32....13
7.1.3. Alimentación del dispositivo....14
7.1.4. Soldadura de las conexiones....14
7.1.5. Carcasa....14
7.1.6. Buzzer....14
7.1.7. Precauciones de uso....14
7.1.8. Documentación de los componentes....14
7.2. Programación....17
7.2.1. Entorno de desarrollo....17
7.2.2. Lectura de los datos del GPS....17
7.2.3. Conexión a internet y envío de datos....17
7.2.4. Servidor....17
7.3. Pruebas y ajustes....18
7.3.1. Prueba de lectura del GPS al aire libre....18
7.3.2. Prueba de envío de datos a la página web....18
7.3.3. Ajustes realizados durante el desarrollo....18
7.4. Registro de actividades....19
7.5. Manual de uso del dispositivo....26
8. Resultados y análisis....27
8.1. Resultados de la lectura del GPS....27
8.2. Resultados del envío de datos y visualización....28

8.3. Resultados del buzzer.....28
8.5. Resultados económicos.....28
9. Conclusiones y recomendaciones.....29
9.1. Conclusiones.....29
10. Bibliografía.....31
11. Anexos.....32
Anexo A: Diagrama de conexión.....32
Conexión de pines entre el módulo GPS NEO-6M (placa GY-GPS6MV2) y el ESP32 DevKit V1 32
Anexo B: Código del ESP32 (Arduino IDE).....32
Anexo C: Captura del monitor serie.....38
Anexo D: Captura de la página web con el mapa.....38

1. Introducción

En la actualidad, la pérdida de mascotas es un problema frecuente en las ciudades, y Asunción no es la excepción. Muchos perros y gatos se escapan de sus casas o se pierden durante los paseos, y sus dueños no tienen forma rápida de saber dónde están. Los collares con placa identificatoria solo sirven si alguien encuentra al animal y decide llamar, lo cual no siempre ocurre.

Con el avance de la tecnología y el abaratamiento de los componentes electrónicos, hoy es posible construir dispositivos de rastreo a un costo accesible. Este proyecto consiste en el desarrollo de un módulo rastreador con GPS para mascotas, cuyo objetivo es permitir conocer la ubicación de un animal desde cualquier celular o computadora con conexión a internet.

El sistema está formado por un microcontrolador ESP32 DevKit V1, un módulo GPS NEO-6M, un servidor desarrollado con Node.js y Express desplegado en la plataforma Render, y una página web con un mapa que muestra la ubicación recibida. El ESP32 lee las coordenadas del GPS, se conecta a internet mediante el hotspot de un celular y envía los datos al servidor, que los almacena y los pone a disposición de la página web.

El presente informe documenta el proceso de trabajo: la investigación previa, la selección de materiales, el armado del circuito, la programación, las pruebas realizadas y los problemas que surgieron, junto con las soluciones aplicadas. Su propósito es dejar constancia del trabajo del equipo y servir como guía para futuros proyectos similares.

2. Planteamiento del problema

2.1 Problemática

La pérdida de mascotas es una situación común en las ciudades. Un perro puede escapar por una puerta abierta, o un gato puede alejarse de su casa y no encontrar el camino de regreso. En estos casos, la familia depende de recorrer las calles, preguntar a los vecinos o publicar avisos en redes sociales, acciones que consumen tiempo y no siempre dan resultado.

Los métodos de identificación más habituales, como las placas en el collar, funcionan únicamente si otra persona encuentra al animal y decide comunicarse con el dueño. No permiten saber dónde se encuentra la mascota en el momento en que desaparece, que es cuando la información resulta más útil.

Frente a esta limitación, la tecnología de posicionamiento por satélite (GPS) ofrece una alternativa: conocer la ubicación del animal de forma directa. Sin embargo, para construir un dispositivo propio es necesario integrar varios elementos, como un módulo GPS, un microcontrolador y un medio para enviar los datos a internet, lo que representa un desafío técnico para estudiantes que trabajan por primera vez con estos componentes.

2.2 Preguntas de investigación

1. ¿Es posible obtener la ubicación de una mascota mediante un módulo GPS NEO-6M conectado a un ESP32?
2. ¿Cómo se conectan físicamente el módulo GPS y el ESP32 para que se comuniquen correctamente?
3. ¿Qué programación necesita el ESP32 para leer las coordenadas del GPS y enviarlas por internet?
4. ¿Cuál es el costo aproximado de construir un prototipo funcional?
5. ¿Qué dificultades surgen durante el armado y la programación, y cómo pueden resolverse?

3. Justificación

El desarrollo de un módulo rastreador con GPS para mascotas se justifica desde tres aspectos.

En primer lugar, desde el punto de vista social, responde a una necesidad real de las familias que tienen animales de compañía. Poder conocer la ubicación de una mascota reduce el tiempo de

búsqueda y la preocupación que genera su pérdida, algo que los métodos tradicionales, como las placas identificatorias, no logran por sí solos.

En segundo lugar, desde el punto de vista educativo, el proyecto permitió al equipo aplicar de forma práctica los contenidos de Robótica. Durante su realización se trabajó con un microcontrolador ESP32, un módulo GPS, comunicación serial (UART), envío de datos por internet mediante solicitudes HTTP, soldadura de componentes y diseño 3D de una carcasa. Aprender estas herramientas en un proyecto concreto resulta más significativo que estudiarlas solo de manera teórica.

En tercer lugar, desde el punto de vista económico y técnico, el prototipo se construyó con componentes de bajo costo y de fácil acceso. Los materiales tuvieron un costo aproximado de Gs. 267.000, incluida la carcasa 3D, financiado con una rifa organizada por el propio equipo, a lo que se sumó la reutilización de cables de proyectos anteriores del colegio, prestados por el profesor. Esto demuestra que es posible desarrollar un dispositivo de rastreo con recursos limitados, y que el diseño puede ampliarse en el futuro con otros módulos, como la comunicación GSM y una batería de mayor capacidad.

Por todo lo anterior, el proyecto es pertinente, viable y útil, tanto por su aplicación práctica como por el aprendizaje que dejó al grupo.

4. Objetivos

4.1 Objetivo general

Diseñar y construir un módulo rastreador con GPS para mascotas, de bajo costo, capaz de obtener la ubicación del animal y enviarla por internet a un servidor propio, para que pueda visualizarse en un mapa desde un celular o una computadora.

4.2 Objetivos específicos

1. Investigar el funcionamiento de los sistemas de posicionamiento global (GPS) y de los módulos disponibles en el mercado local.
2. Seleccionar los componentes adecuados según su precio, disponibilidad y facilidad de uso, priorizando el ESP32 DevKit V1 y el módulo GPS NEO-6M.
3. Reunir los fondos necesarios mediante una rifa organizada por el equipo y calcular el costo de los componentes del prototipo.
4. Practicar la técnica de soldadura y soldar los cables de conexión del prototipo.
5. Realizar el cableado y la conexión física entre el módulo GPS y el microcontrolador ESP32.
6. Programar el ESP32 en el entorno Arduino IDE para que lea los datos de latitud, longitud y cantidad de satélites a través del puerto Serial2.
7. Configurar la conexión del ESP32 a internet mediante el hotspot de un teléfono celular.
8. Desarrollar un servidor con Node.js y Express que reciba las coordenadas mediante una petición POST y las almacene.
9. Desplegar el servidor en la plataforma Render y verificar su funcionamiento en línea.

10. Crear una página web con un mapa que muestre la ubicación recibida desde el servidor.
11. Alimentar el dispositivo con pilas AA de 1,5 V, un portapilas y un interruptor de encendido.
12. Diseñar una carcasa en 3D para alojar los componentes en el collar de la mascota.

5. Marco teórico

5.1 Antecedentes

Los rastreadores GPS construidos con microcontroladores y módulos de bajo costo son un tema frecuente en proyectos de electrónica y en la investigación. Un ejemplo es un proyecto publicado en Hackster.io que utiliza un módulo GPS Neo-6M junto con una placa XIAO ESP32-S3. Ese rastreador lee los datos NMEA del GPS por UART y envía las coordenadas por Wi-Fi a una plataforma en la nube, sin tarjeta SIM ni servicio celular (CircuitDigest, 2025). Para interpretar los datos del GPS utiliza la librería TinyGPSPlus, la misma que se empleó en el presente trabajo.

En el ámbito académico, Mohd Zahari y Baharuddin (2023) desarrollaron un sistema de rastreo y monitoreo de salud de mascotas al aire libre. El sistema combina un ESP32, un módulo GPS Neo-6M y sensores de movimiento y temperatura, se programa con el Arduino IDE y muestra los datos en la aplicación Blynk. Los autores concluyen que el prototipo se desarrolló con éxito y que, para funcionar bien, requiere una buena conexión a internet.

Por su parte, Prasojo et al. (2026) diseñaron un sistema de rastreo de mascotas con un módulo GPS Neo-6M, comunicación LoRa y notificaciones por Telegram; según sus pruebas, el sistema envía datos de ubicación precisos al dueño.

También existen proyectos orientados a collares inteligentes, como IntelliCollar, un collar basado en ESP32 que mide el movimiento y los latidos del animal, obtiene su ubicación GPS con la librería TinyGPS++ y la muestra en un panel web (Gupta, 2025).

El presente proyecto coincide con estos antecedentes en el uso de un ESP32, un módulo GPS Neo-6M, el entorno Arduino IDE y una librería para interpretar los datos del GPS. Se diferencia en que los datos se envían a un servidor propio, desarrollado con Node.js y Express y desplegado en Render, y en que el equipo lo construyó con materiales reutilizados y de bajo costo, centrado únicamente en la ubicación de la mascota.

5.2 Bases teóricas

5.2.1 Sistema de posicionamiento global (GPS)

El GPS es un sistema de navegación que permite conocer la posición de un receptor en cualquier punto de la Tierra mediante señales enviadas por satélites. Cada satélite transmite su posición y la hora exacta en que emite la señal. El receptor mide el tiempo que tarda cada señal en llegar y, con ello, calcula su distancia a cada satélite. Para obtener latitud, longitud y altitud se necesitan las

señales de al menos cuatro satélites. Por esta razón el GPS funciona mejor en exteriores, con vista libre del cielo, y puede fallar o tardar más en espacios cerrados.

5.2.2 Módulo GPS NEO-6M

El NEO-6M es un receptor GPS fabricado por u-blox. En este proyecto se utilizó en la placa GY-GPS6MV2, que incorpora una antena cerámica y facilita su conexión a un microcontrolador. El módulo se comunica por UART a 115200 baudios y entrega la posición una vez por segundo (u-blox, s.f.). En condiciones favorables su precisión es de unos pocos metros.

El módulo NEO-6M viene configurado de fábrica a 9600 baudios. Para este proyecto, el equipo lo configuró a 115200 baudios enviándole el comando $PUBX,41,1,0007,0003,115200,0*1E desde el ESP32 al iniciar el programa, y luego reinicializando el puerto Serial2 a esa velocidad.

5.2.3 Microcontrolador ESP32

El ESP32 es un microcontrolador desarrollado por Espressif Systems. Cuenta con dos núcleos de procesamiento, conectividad Wi-Fi y Bluetooth integrada, y varios puertos UART (Espressif Systems, s.f.). En el proyecto se usó la placa ESP32 DevKit V1, que añade un puerto USB para programarla y pines de entrada y salida. Su Wi-Fi permite enviar los datos por internet sin un módulo adicional. Para leer el GPS se utilizó el puerto Serial2, configurado en los pines 18 (recepción) y 5 (transmisión).

5.2.4 Comunicación serial UART

UART es un tipo de comunicación serial asíncrona entre dos dispositivos que usa dos líneas: transmisión (TX) y recepción (RX). La conexión es cruzada: el TX de un dispositivo se conecta al RX del otro. Ambos deben usar la misma velocidad (baudios) y el mismo formato. En este proyecto se usó 115200 baudios con formato 8N1 (8 bits de datos, sin paridad y 1 bit de parada), declarado en el código como Serial2.begin(115200, SERIAL_8N1, 18, 5).

5.2.5 Mensajes NMEA y la librería TinyGPSPlus

El módulo GPS envía su información en mensajes de texto con el estándar NMEA, que incluyen la hora, las coordenadas y la cantidad de satélites. Interpretarlos directamente es complicado, por eso se utilizó la librería TinyGPSPlus, que los decodifica y entrega valores como latitud, longitud y satélites (Hart, s.f.).

5.2.6 Protocolo HTTP y método POST

HTTP es el protocolo con el que se comunican los clientes y los servidores en internet. El método POST permite que un cliente envíe datos al servidor dentro del cuerpo de la solicitud. En el proyecto, el ESP32 actúa como cliente y envía las coordenadas mediante POST al servidor, que las recibe y las almacena.

5.2.7 Node.js, Express y Render

Node.js es un entorno que permite ejecutar JavaScript fuera del navegador, y Express es un marco de trabajo para crear servidores web de forma sencilla. Con ellos se desarrolló el servidor que recibe las coordenadas. El código se guarda en un repositorio de GitHub y se despliega en Render, una plataforma en la nube que lo publica en internet (Render, s.f.).

5.2.8 Alimentación eléctrica con pilas

Inicialmente se probó con dos pilas recargables de tipo AAA (1,2 V cada una), pero la energía no alcanzaba para encender el módulo GPS. Por eso se reemplazaron por pilas AA de 1,5 V, que entregan mayor voltaje. Al conectarlas en serie, sus voltajes se suman, y un interruptor intercalado permite encender y apagar el dispositivo sin desconectar los cables.

5.2.9 Soldadura y diseño 3D

La soldadura une dos piezas metálicas con estaño fundido y crea conexiones eléctricas firmes, más seguras que los contactos sueltos. Para la carcasa se utiliza diseño 3D asistido por computadora, con el fin de imprimir una caja que proteja los componentes dentro del collar.

5.3 Definición de términos

Baudios: unidad que mide la velocidad de transmisión de datos en una comunicación serial.

Buzzer activo: componente que emite un tono fijo al recibir corriente eléctrica.

Coordenadas: valores de latitud y longitud que indican una posición en la Tierra.

ESP32: microcontrolador con Wi-Fi y Bluetooth integrados.

GitHub: plataforma donde se guarda y comparte el código de un proyecto.

GPS: sistema que determina la posición de un receptor mediante señales de satélites.

Hotspot: función del celular que comparte su conexión a internet con otros dispositivos.

HTTP: protocolo de comunicación entre clientes y servidores en internet.

Latitud: distancia angular de un punto al norte o al sur del ecuador.

Longitud: distancia angular de un punto al este o al oeste del meridiano de Greenwich.

Microcontrolador: pequeño circuito programable que controla otros componentes.

NMEA: estándar de mensajes de texto que utilizan los módulos GPS.

Node.js / Express: entorno y marco de trabajo para crear servidores con JavaScript.

POST: método HTTP para enviar datos a un servidor.

Portapilas: soporte que sujeta las pilas y las conecta al circuito.

Render: plataforma en la nube donde se publica el servidor.

Servidor: computadora o programa que recibe, guarda y entrega información a otros dispositivos.

UART: comunicación serial entre dos dispositivos mediante líneas TX y RX.

6. Marco metodológico

6.1 Tipo de investigación

El presente trabajo es un proyecto tecnológico de desarrollo aplicado. Su finalidad es construir un prototipo funcional que dé solución a un problema concreto: conocer la ubicación de una mascota.

El enfoque es práctico y experimental, porque el equipo armó el circuito, programó el microcontrolador, realizó pruebas y corrigió los errores a medida que aparecían. Además, tiene un componente descriptivo, ya que el proceso completo se registró en una bitácora y en este informe.

6.2 Diseño técnico

El sistema se diseñó con la siguiente arquitectura:

1. El módulo GPS NEO-6M (placa GY-GPS6MV2) obtiene la latitud y la longitud a partir de las señales de los satélites.
2. El ESP32 DevKit V1 lee esos datos por el puerto Serial2.
3. El ESP32 se conecta a internet mediante el hotspot de un teléfono celular y envía las coordenadas por el método POST.
4. Un servidor desarrollado con Node.js y Express, desplegado en Render, recibe y almacena las coordenadas. El código está guardado en un repositorio de GitHub.
5. Una página web con un mapa muestra la ubicación recibida desde el servidor, desde un celular o una computadora.

La conexión física entre el módulo GPS y el ESP32 es la siguiente:

Pin del GPS NEO-6M	Color del cable	Pin del ESP32
TX	Marrón	D18 (RX2)
RX	Negro	D5 (TX2)
VCC	Blanco	VIN
GND	Rojo	GND

El puerto Serial2 se inicializa en el código con la instrucción Serial2.begin(115200, SERIAL_8N1, 18, 5);, donde 18 corresponde al pin de recepción (RX) y 5 al de transmisión (TX). La conexión entre TX y RX es cruzada: el TX del GPS va al RX del ESP32, y viceversa.

6.3 Materiales y métodos

Material	Cantidad	Origen	Costo (Gs.)
ESP32 DevKit V1 con cable USB	1	Comprado	65.000
Cable USB nuevo (reemplazado)	1	Comprado	20.000
Módulo GPS NEO-6M (GY-GPS6MV2)	1	Comprado	35.000
Pilas AAA recargables (1,2 V), portapilas e interruptor -. Primer intento, no funcionó	2 pilas, 1 portapilas, 1 interruptor	Aportados por el profesor	62.000

Material	Cantidad	Origen	Costo (Gs.)
Portapila nueva (reemplazado)	1	Reemplazado por el anterior	Sin costo
Pilas AA (1,5 V) - Definitivas	2	Comprado	5.000
Carcasa 3D (diseño e impresión)	1	Diseñada por el profesor	80.000
Cables de conexión	9 utilizados, de unos 20 a 25 disponibles	Reciclados de proyectos anteriores del colegio; 4 son Dupont hembra-hembra aportados por el profesor	Sin costo
Buzzer	1	Reciclado de proyectos anteriores del colegio	Sin costo
Total			**267.000**

El ESP32 y el módulo GPS se adquirieron a un precio reducido gracias al descuento de un contacto del profesor. La carcasa 3D fue diseñada e impresa por encargo del profesor y tuvo un costo de Gs. 80.000, incluido en el total. Los cables de conexión y el buzzer fueron reutilizados de proyectos anteriores del colegio, por lo que no representaron un gasto.

Inicialmente se utilizó un portapilas para dos pilas AAA, pero como no alcanzaba para encender el módulo GPS, el profesor lo cambió por un portapilas de mayor capacidad para pilas AA, sin costo adicional para el equipo.

6.3.2 Herramientas y software

- Equipo de soldadura, para fijar las conexiones del circuito.
- Arduino IDE 2.3.10, entorno de programación del ESP32.
- Paquete de placas “esp32 by Espressif Systems”, versión 3.3.12.
- Librería TinyGPSPlus, versión 1.0.3, para interpretar los datos del GPS.
- Node.js y Express, para el servidor.
- GitHub y Render, para guardar y publicar el servidor.
- Tinkercad, para el diseño 3D de la carcasa, recomendado por el profesor.

6.3.3 Métodos de trabajo

El proyecto se desarrolló en equipo, con tareas repartidas según las posibilidades de cada integrante:

- **Daiana Ríos:** investigación de los componentes, sus conexiones y sus precios; armado del circuito; registro de la bitácora.
- **Marina Sugastti:** armado de los componentes y redacción de la documentación.
- **Marcos Santacruz:** programación del proyecto.
- **Giovanni Peralta y José Spina:** intento de diseño 3D de la carcasa y colaboración en las pruebas del GPS al aire libre. Las herramientas de diseño que probaron eran de pago, por lo que no lograron completar la carcasa. El profesor recomendó Tinkercad y finalmente se encargó de realizar el diseño.

El armado de los componentes se realizó con la ayuda del profesor, quien además enseñó a soldar. El trabajo siguió estas etapas:

1. Elección del tema, mediante una lista de pros y contras y una votación.
2. Obtención de fondos, con una rifa organizada por el equipo.
3. Investigación y selección de los componentes, con la aprobación del profesor.
4. Compra y recepción de los materiales.
5. Práctica de soldadura y soldadura de las conexiones.
6. Armado del circuito entre el GPS y el ESP32.
7. Programación del ESP32 y del servidor.
8. Pruebas y corrección de errores.
9. Registro diario en la bitácora y redacción de este informe.

7. Desarrollo del proyecto

7.1. Diseño del hardware

7.1.1. Componentes principales

El hardware del prototipo está formado por un microcontrolador ESP32 DevKit V1, un módulo GPS NEO-6M (placa GY-GPS6MV2), un portapilas con pilas AA, un interruptor de encendido y un buzzer activo. Las conexiones se realizaron con cables soldados y cables Dupont, reutilizados de proyectos anteriores del colegio y prestados por el profesor.

El ESP32 procesa los datos y los envía por internet, mientras que el módulo GPS obtiene la latitud y la longitud a partir de las señales de los satélites. Las pilas y el interruptor permiten que el dispositivo funcione sin estar conectado a una computadora, y el buzzer sirve como alerta sonora.

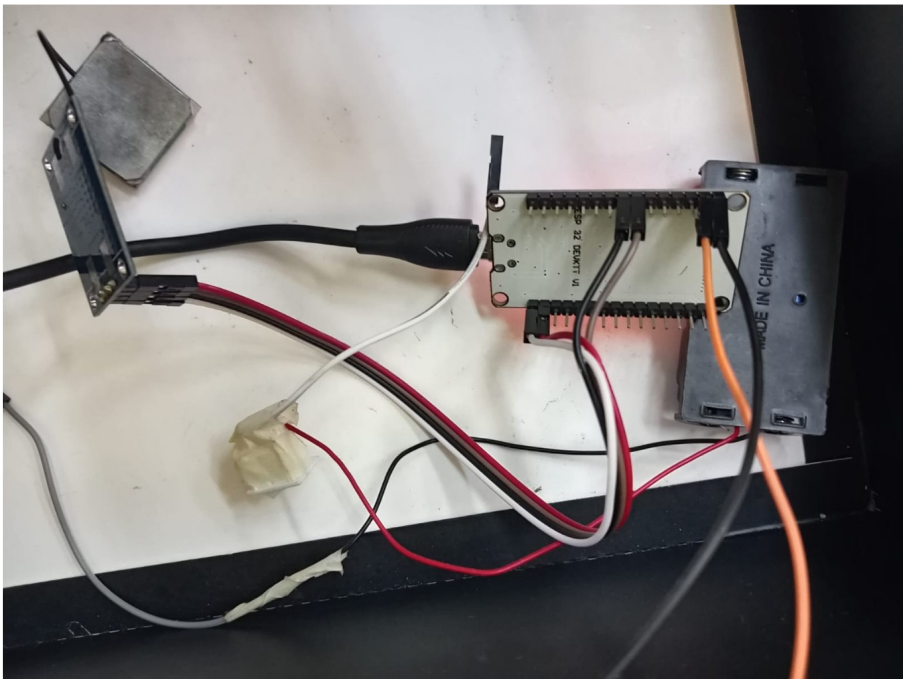
7.1.2. Conexión del módulo GPS con el ESP32

El módulo GPS se conectó al ESP32 mediante comunicación serial UART, utilizando el puerto Serial2 de la placa. Los pines se conectaron de la siguiente manera:

Conexión de pines entre el GPS NEO-6M y el ESP32 DevKit V1

Pin del GPS NEO-6M	Color del cable	Pin del ESP32	Función
TX	Marrón	D18 (RX2)	Datos del GPS hacia el ESP32
RX	Negro	D5 (TX2)	Datos del ESP32 hacia el GPS
VCC	Blanco	VIN	Alimentación
GND	Rojo	GND	Tierra común

La conexión de los pines de datos es cruzada: el TX del GPS se conecta al RX del ESP32, y el RX del GPS al TX del ESP32. Esto es necesario para que lo que transmite un dispositivo sea recibido por el otro. Los dos dispositivos comparten además la tierra (GND), que sirve de referencia común para las señales.



7.1.3. Alimentación del dispositivo

El dispositivo puede alimentarse de dos maneras: mediante el cable USB, durante la programación y las pruebas, o con pilas AA alojadas en un portapilas, para su uso sin computadora. En el segundo caso, el polo positivo del portapilas se conecta, a través de un interruptor, al pin VIN del ESP32, y el polo negativo al pin GND. El módulo GPS recibe su energía del mismo pin VIN.

Inicialmente se probó con dos pilas recargables de tipo AAA (de 1,2 V cada una), pero la energía no alcanzaba para encender el módulo GPS. Por eso se reemplazaron por pilas AA de 1,5 V, que entregan mayor voltaje. Al conectarlas en serie, sus voltajes se suman, y el interruptor intercalado permite encender y apagar el dispositivo sin desconectar los cables. En las pruebas realizadas, el ESP32 y el módulo GPS encendieron correctamente con esta alimentación.

7.1.4. Soldadura de las conexiones

Para lograr conexiones firmes y evitar falsos contactos, las uniones del circuito se soldaron. Antes de trabajar con los componentes, el equipo practicó con cables sueltos, para no dañar las placas durante el aprendizaje. Esta actividad se realizó con la guía del profesor, quien enseñó la técnica. Una vez adquirida la práctica, se soldaron los cables de las conexiones del prototipo.

7.1.5. Carcasa

Para proteger los componentes y poder colocarlos en el collar de la mascota, el profesor diseñó una carcasa en 3D. Consta de una base y una tapa unidas mediante tornillos, con una orejeta lateral para pasar la correa del collar. El diseño y la impresión tuvieron un costo de Gs. 80.000.

7.1.6. Buzzer

Se incorporó un buzzer activo, reutilizado de proyectos anteriores del colegio. A diferencia del buzzer pasivo, el activo tiene un circuito interno que produce un tono fijo con solo recibir voltaje, sin necesidad de que el microcontrolador envíe una señal de frecuencia.

Su función es servir de alerta de salida de zona: se define un punto central (la casa) y un radio de zona permitida; si la mascota sale de ese radio, el buzzer emite una serie de pitidos para avisar al dueño. Al encender, el dispositivo hace un pitido breve que confirma que el sistema funciona. Se conectó al ESP32 en los pines D23 (positivo) y D22 (negativo), tal como quedó definido en la programación del equipo: al encender, el programa activa ambos pines para emitir el tono.

7.1.7. Precauciones de uso

Durante el armado y las pruebas se siguieron las siguientes precauciones: utilizar una sola fuente de energía a la vez (USB o pilas, nunca ambas), apagar el dispositivo antes de mover los cables y verificar la polaridad de las pilas antes de encenderlo, ya que invertirla puede dañar la placa.

7.1.8. Documentación de los componentes

El prototipo está formado por ocho componentes principales. A continuación se describe cada uno con su función dentro del sistema y sus características técnicas.



ESP32 DevKit V1

Microcontrolador de 32 bits de Espressif Systems, con Wi-Fi y Bluetooth integrados. Es el cerebro del sistema: lee los datos del GPS por el puerto Serial2, se conecta a internet y envía las coordenadas al servidor. Funciona con 3,3 V y se alimenta por USB o por el pin VIN.



Módulo GPS NEO-6M (GY-GPS6MV2)

Receptor GPS basado en el chip u-blox NEO-6M. Obtiene latitud, longitud, hora UTC y cantidad de satélites a partir de las señales satelitales. Incluye antena cerámica y batería de respaldo. Se comunica por UART y entrega la posición una vez por segundo.



Buzzer activo

Componente que emite un tono fijo al recibir corriente, sin necesidad de una señal de frecuencia. En el proyecto funciona como alerta sonora cuando la mascota sale del radio de zona permitida. Se conectó a los pines D23 y D22 del ESP32.



Portapilas

Soporte que aloja las pilas y las conecta al circuito. Permite que el dispositivo funcione sin estar conectado a una computadora.



Pilas AA de 1,5 V

Fuente de energía del dispositivo. Se colocan en serie para sumar su voltaje. Reemplazaron a las pilas AAA recargables de 1,2 V, que no alcanzaban para encender el módulo GPS.



Interruptor de encendido

Permite encender y apagar el dispositivo sin desconectar los cables. Está intercalado entre el polo positivo del portapilas y el pin VIN del ESP32.



Cables de conexión

Cables soldados y cables Dupont que unen los componentes. Se reutilizaron de proyectos anteriores del colegio y fueron prestados por el profesor.



Carcasa 3D

Caja impresa en 3D que protege los componentes y permite sujetar el dispositivo al collar. Consta de una base y una tapa unidas con tornillos, con una orejeta lateral para pasar la correa.

7.2. Programación

7.2.1. Entorno de desarrollo

El ESP32 se programó en el entorno Arduino IDE (versión 2.3.10), con el paquete de placas "esp32 by Espressif Systems" (versión 3.3.12). Para interpretar los datos del módulo GPS se utilizó la librería TinyGPSPlus (versión 1.0.3). El código se desarrolló con apoyo de una herramienta de inteligencia artificial y luego fue revisado y probado por el equipo. Durante el proceso fue necesario reescribir el programa del ESP32 desde cero, porque el módulo GPS se había utilizado en otro dispositivo y los códigos se habían modificado.

7.2.2. Lectura de los datos del GPS

El módulo GPS se comunica con el ESP32 por el puerto Serial2, que se inicializa con la instrucción Serial2.begin(115200, SERIAL_8N1, 18, 5). Sus parámetros indican una velocidad de 115200 baudios, el formato 8N1 (8 bits de datos, sin paridad y 1 bit de parada), el pin 18 como recepción (RX) y el pin 5 como transmisión (TX).

Como el NEO-6M viene de fábrica a 9600 baudios, el programa primero inicia el puerto a esa velocidad, envía el comando $PUBX,41,1,0007,0003,115200,0*1E para cambiar el módulo a 115200 y luego reinicia el puerto Serial2 a la nueva velocidad. De esta forma, el ESP32 y el GPS quedan comunicándose a la misma velocidad.

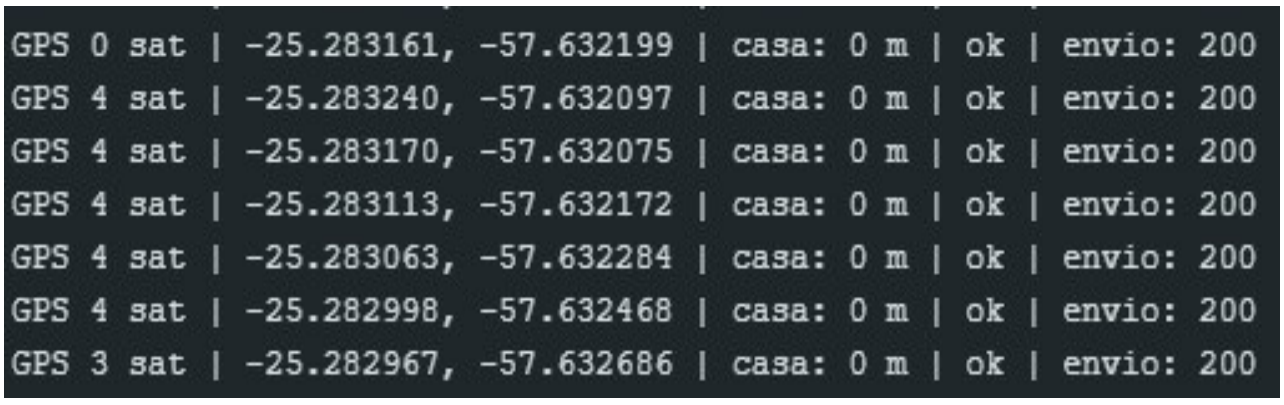
Los mensajes NMEA que envía el GPS son decodificados por la librería TinyGPSPlus, que permite obtener la latitud, la longitud y la cantidad de satélites detectados. Estos valores se muestran en el monitor serie del Arduino IDE, lo que permitió verificar que el módulo recibía señal de los satélites.

7.2.3. Conexión a internet y envío de datos

El ESP32 se conecta a internet mediante el hotspot de un teléfono celular, usando su Wi-Fi integrado. Una vez conectado, el programa envía las coordenadas al servidor mediante una solicitud HTTP con el método POST, cada 5 segundos. El código completo del ESP32 se presenta en el Anexo B.

7.2.4. Servidor

El servidor se desarrolló con Node.js y Express. Recibe las coordenadas enviadas por el ESP32 y las almacena, y está desplegado en la plataforma Render, con su código guardado en un repositorio de GitHub. La página web consulta el servidor y muestra la última ubicación recibida en un mapa.



7.3. Pruebas y ajustes

7.3.1. Prueba de lectura del GPS al aire libre

La prueba se realizó al aire libre, en el Colegio Nacional E.M.D. Asunción Escalada (Asunción, Paraguay). Se alimentó el dispositivo, se esperó a que el módulo GPS detectara satélites y se observaron los datos en el monitor serie del Arduino IDE. Los resultados se resumen en la siguiente tabla:

Aspecto evaluado	Resultado
Lugar	Colegio Nacional E.M.D. Asunción Escalada, al aire libre
Tiempo hasta obtener señal	Aproximadamente 30 minutos
Satélites detectados	6
Coincidencia de las coordenadas con el lugar real	Sí

El módulo logró obtener coordenadas que coincidían con la ubicación real, lo que confirma que la conexión con el ESP32 y la lectura de los datos funcionaron correctamente. El tiempo de espera fue prolongado; la primera conexión de un módulo GPS suele tardar más, y la señal puede verse afectada por edificios, techos y otros obstáculos que reducen la cantidad de satélites visibles.

7.3.2. Prueba de envío de datos a la página web

Una vez confirmada la lectura del GPS, se probó el envío de las coordenadas al servidor. El ESP32 se conectó al hotspot de un celular y comenzó a enviar los datos por el método POST. La página web recibió y mostró correctamente la ubicación en el mapa, lo que confirmó que todo el sistema — GPS, ESP32, servidor y página web— funciona de forma integrada.

7.3.3. Ajustes realizados durante el desarrollo

A lo largo del proyecto fue necesario corregir varios problemas técnicos:

Incompatibilidad del módulo GPS con el dispositivo de la institución: se reprogramó el ESP32 desde cero y se revisaron los pines.
Código modificado: como el módulo GPS se usó en otro dispositivo y los códigos se habían cambiado, se reescribió el programa completo.
Cable USB defectuoso: el cable original no funcionaba, por lo que se adquirió uno nuevo para cargar el programa.
Alimentación insuficiente: las pilas AAA de 1,2 V no daban la energía necesaria, por lo que se reemplazaron por pilas AA de 1,5 V.
Envío de datos a la página web: durante el desarrollo la página no recibía las coordenadas; se revisaron las conexiones y el envío de datos hasta lograr que funcionara.
Diseño 3D de la carcasa: el equipo no logró completarlo con las herramientas que probó; el profesor recomendó Tinkercad y se encargó del diseño.

7.4. Registro de actividades

BITÁCORA DE DESARROLLO DEL MODULO RASTREADOR GPS PARA MASCOTAS

A continuación se presenta el registro cronológico y sistemático de las actividades desarrolladas por los estudiantes durante el diseño, programación, ensamblaje y preparación del prototipo de monitoreo.

Día 1 — 02/07/2026

Objetivo del día: Conformar el equipo de trabajo, conocer la propuesta del proyecto y repartir los roles de cada integrante.

Actividades realizadas:

El profesor nos presentó la propuesta de trabajo del módulo de robótica para el año.
Conversamos sobre las ideas que el profe nos dio para el proyecto y elegimos la que nos pareció más útil y más viable.
Decidimos hacer un módulo rastreador con GPS para mascotas, pensando en la cantidad de perros y gatos que se pierden en Asunción.
Una vez definido el proyecto, repartimos los roles de cada integrante:
Daiana Ríos: investigación de los componentes, sus conexiones y sus precios; armado del circuito; registro de la bitácora.
Marina Sugastti: armado de los componentes y redacción de la documentación.
Marcos Santacruz: programación del proyecto.
Giovanni Peralta y José Spina: diseño 3D de la carcasa y colaboración en las pruebas del GPS al aire libre.

Anotamos los materiales que creíamos que íbamos a necesitar y las dudas que teníamos.
Problemas encontrados: Al principio no nos decidíamos entre varios temas; algunos nos parecían muy difíciles y otros muy simples. Además, no sabíamos bien qué componentes electrónicos existían para hacer un rastreador.

Soluciones aplicadas: Hicimos una lista de pros y contras de cada idea y votamos. Elegimos el rastreador GPS porque combina programación, electrónica y una utilidad real. Para las dudas de materiales, quedamos en investigar en casa y traer la información la próxima clase.

Conclusión del día: El equipo quedó conformado y con el tema definido. Tenemos una idea clara de lo que queremos lograr y una lista inicial de tareas para la siguiente clase.

Día 2 — 08/07/2026

Objetivo del día: Buscar la forma de conseguir el dinero necesario para comprar los componentes del proyecto.

Actividades realizadas:

Conversamos en equipo sobre distintas maneras de recaudar fondos para poder comprar todo lo que necesitamos.
Evaluamos varias ideas y decidimos organizar una rifa.
Cada integrante se comprometió a aportar un premio para la rifa.

Cada uno llevaría 20 números para vender a Gs. 10.000 cada uno.
Nos fijamos como meta recaudar un millón de guaraníes.

Problemas encontrados: Al principio no nos poníamos de acuerdo sobre qué actividad hacer para juntar el dinero, porque algunas opciones requerían inversión previa o mucho tiempo.

Soluciones aplicadas: Elegimos la rifa porque es una actividad sencilla, no requiere mucho dinero para arrancar y cada uno podía aportar un premio sin grandes gastos. Repartimos los números de forma equitativa entre todos los integrantes.

Conclusión del día: Quedó definida la rifa como estrategia de recaudación y logramos alcanzar la meta de un millón de guaraníes, con lo cual ya contamos con el presupuesto para comprar los componentes del proyecto.

Día 3 — 09/07/2026

Objetivo del día: Investigar los componentes necesarios para el proyecto, confirmar la lista con el profesor y realizar la compra.

Actividades realizadas:

Investigamos entre todos cuáles eran los componentes que íbamos a necesitar para armar el rastreador.
Buscamos información sobre cada componente y su función dentro del sistema.
Armamos, entre todos, unas imágenes de los componentes principales para tener claro cómo eran.
Consultamos con el profesor para que nos confirmara si la lista de componentes estaba bien y si faltaba algo.
Con su visto bueno, hicimos la compra del ESP32 con su cable y del módulo GPS.

Problemas encontrados: No sabíamos con certeza si los componentes que habíamos elegido eran los correctos o si hacía falta algún otro material para el armado.

Soluciones aplicadas: Antes de comprar, le consultamos al profesor, quien revisó nuestra lista y nos confirmó que estaba bien. Recién después de su aprobación realizamos la compra del ESP32, el cable y el módulo GPS.

Conclusión del día: Quedó hecha la compra de los componentes principales con la aprobación del profesor. A partir de ese momento estuvimos aproximadamente dos semanas esperando a que llegaran los materiales para poder iniciar el armado del circuito.

Día 4 — 29/07/2026

Objetivo del día: Recibir los componentes comprados y verificar que estuvieran completos para poder iniciar el armado.

Actividades realizadas:

Recibimos los componentes que habíamos comprado: el ESP32 con su cable y el módulo GPS.
El profesor nos prestó varios cables para las conexiones, que provenían de proyectos anteriores del colegio.
Revisamos que todos los materiales estuvieran en condiciones.
Aún no realizamos ninguna conexión; solo organizamos el material recibido.

Problemas encontrados: No tuvimos mayores inconvenientes, ya que los componentes llegaron completos y en buen estado.

Soluciones aplicadas: No fue necesario aplicar ninguna solución, porque el material recibido estaba en condiciones para ser utilizado.

Conclusión del día: Ya tenemos todos los componentes necesarios para comenzar con el armado del circuito. El siguiente paso será realizar las conexiones entre el ESP32 y el módulo GPS.

Día 5 — 30/07/2026

Objetivo del día: Aprender a soldar los cables y realizar las primeras conexiones con el ESP32.

Actividades realizadas:

El profesor nos llamó para enseñarnos a soldar.
Antes de tocar nuestros componentes, practicamos la soldadura con cables sueltos, para no descomponer ni hacer algo mal con el ESP32 y el módulo GPS.
Una vez que manejamos mejor la técnica, soldamos los cables que se iban a usar para las conexiones del circuito.
Problemas encontrados: Al ser la primera vez que soldábamos, nos costó al principio controlar la temperatura del cautín y evitar que el estaño se acumulara de más o que los cables se movieran mientras soldábamos.

Soluciones aplicadas: Para no arriesgar los componentes, primero practicamos con cables sueltos. El profesor nos guió paso a paso, y repitiendo varias veces logramos soldar los cables de forma firme y prolija.

Conclusión del día: Aprendimos a soldar y dejamos listas las conexiones de los cables con el ESP32, un paso fundamental para poder armar el circuito del rastreador.

Día 6 — 06/08/2026

Objetivo del día: Realizar las conexiones entre el ESP32 y el módulo GPS, cargar el programa para buscar señal de satélite y recibir el desafío de la página web.

Actividades realizadas:

El profesor nos volvió a llamar y nos pasó los componentes.
Realizamos las conexiones entre el ESP32 y el módulo GPS.
Cargamos los códigos en el ESP32 para que empezara a buscar un satélite y conectarse al GPS.
El profesor nos puso como desafío hacer la página web, parecida a un Google Maps, que estuviera conectada al GPS.
El objetivo era que las señales de ubicación aparecieran en esa página.
Problemas encontrados: Aún no sabíamos cómo hacer que la ubicación del GPS llegara a una página web, ya que era la primera vez que íbamos a trabajar con un mapa en línea.

Soluciones aplicadas: Anotamos el desafío y quedamos en investigar cómo conectar el ESP32 con un servidor y una página web para mostrar las coordenadas en un mapa.

Conclusión del día: El circuito quedó conectado y el ESP32 cargado con el programa y GPS para buscar señal de satélite. Nos llevamos el desafío de crear la página web con el mapa conectado al GPS.

Día 7 — 12/08/2026

Objetivo del día: Avanzar con la programación del ESP32 y comenzar a trabajar en la página web conectada al GPS.

Actividades realizadas:

Cargamos y probamos el código en el ESP32 para que leyera los datos del módulo GPS.
Verificamos que el módulo GPS recibiera las señales de los satélites.
Empezamos a trabajar en la página web con el mapa, que debía mostrar la ubicación recibida del GPS.
Investigamos cómo conectar el ESP32 con el servidor para enviar las coordenadas.

Problemas encontrados: Nos costaba lograr que la ubicación del GPS apareciera en la página web, ya que era la primera vez que trabajábamos con un mapa en línea.

Soluciones aplicadas: Pedimos ayuda para programar el ESP32 y para armar la página web con el mapa, y fuimos probando paso a paso hasta que las señales de ubicación se reflejaron en la página.

Conclusión del día: Logramos avanzar en la programación y dimos los primeros pasos para conectar la ubicación del GPS con la página web.

Día 8 — 13/08/2026

Objetivo del día: Continuar con la programación del ESP32 y la página web, y buscar soluciones a los problemas que impedían su funcionamiento.

Actividades realizadas:

Seguimos trabajando con el ESP32 y el módulo GPS, ya que el sistema todavía no funcionaba como esperábamos.
Revisamos el código cargado en el ESP32 para detectar posibles errores.
Probamos distintas alternativas para que la ubicación del GPS llegara correctamente a la página web.
Volvimos a probar las conexiones y el envío de datos.

Problemas encontrados: El sistema seguía sin funcionar: la ubicación no llegaba a mostrarse en la página web como necesitábamos.

Soluciones aplicadas: Fuimos revisando paso por paso, probando cambios en el código y en las conexiones, y buscando información para entender dónde estaba el error.

Conclusión del día: Aunque el sistema todavía no funcionaba, identificamos mejor dónde estaban las fallas y quedamos con varias ideas para seguir probando en los próximos días.

Día 9 — 19/08/2026

Objetivo del día: Retomar el trabajo en la institución, revisar el estado del módulo GPS y continuar con la página web.

Actividades realizadas:

Tuvimos que empezar desde cero la programación del ESP32, porque el módulo GPS se había llevado a otro dispositivo y los códigos se habían cambiado.
Al volver a la institución para continuar el trabajo, encontramos errores en el módulo GPS.
Detectamos que el módulo GPS no era compatible con el dispositivo de la institución.
Terminamos la página web, que seguía sin recibir las coordenadas del GPS.

Problemas encontrados: El módulo GPS había sido usado en otro dispositivo y los códigos estaban modificados, así que hubo que reprogramar todo. Además, el módulo no resultó compatible con el dispositivo de la institución, y la página web seguía sin recibir las coordenadas.

Soluciones aplicadas: Volvimos a programar el ESP32 desde cero y dejamos la página web terminada, a la espera de resolver el problema de la recepción de coordenadas.

Conclusión del día: La página web quedó terminada, pero seguía sin recibir las coordenadas del GPS. Quedamos pendientes de solucionarlo otro día.

Día 10 — 26/08/2026

Objetivo del día: Avanzar en paralelo con el diseño 3D del collar, la actualización de la bitácora y la documentación, y la solución del problema de la página web.

Actividades realizadas:

Los compañeros Giovanni y José estuvieron intentando hacer el diseño 3D del collar, pero tuvieron problemas porque los diseños no les salían como esperaban.
Fueron a consultar con el profesor, y él les recomendó que intentaran en Tinkercad, una aplicación gratuita para hacer este tipo de diseños.
Consultamos con el profesor si tenía disponibles pilas, portapilas e interruptor para el funcionamiento del dispositivo.
Las compañeras Daiana y Marina estuvieron modificando y actualizando la bitácora y la documentación, haciéndolas más formales, porque las que teníamos eran muy resumidas.
El compañero Marcos estuvo tratando de solucionar el problema de la página web con el GPS.

Problemas encontrados: A Giovanni y José no les salía el diseño 3D del collar. Además, la página web seguía sin funcionar porque el GPS no enviaba las señales de ubicación.

Soluciones aplicadas: El profesor les recomendó Tinkercad para el diseño 3D y le consultamos por las pilas, el portapilas y el interruptor necesarios para alimentar el dispositivo. Mientras tanto, Daiana y Marina mejoraron la bitácora y la documentación, y Marcos siguió investigando el problema del envío de coordenadas.

Conclusión del día: El equipo trabajó en varios frentes a la vez: el diseño 3D del collar, la gestión de los componentes de alimentación, la mejora de la documentación y la solución del problema del GPS con la página web. Quedó pendiente resolver el envío de las señales de ubicación.

Día 11 — 02/09/2026

Objetivo del día: Resolver los problemas de conexión y de alimentación que impedían el funcionamiento del prototipo.

Actividades realizadas:

Detectamos que el cable USB original ya no funcionaba: la computadora no reconocía la placa y el programa no se cargaba.

Compramos un cable USB nuevo y verificamos que el ESP32 se conectara correctamente y que el código se pudiera cargar sin problemas.

Probamos el dispositivo con las pilas AAA recargables de 1,2 V, pero la energía no alcanzaba para encender el módulo GPS.

Decidimos reemplazarlas por pilas AA de 1,5 V, que entregan mayor voltaje.

Como el portapilas que teníamos era para dos pilas AAA, el profesor nos lo cambió por uno de mayor capacidad para pilas AA.

Volvimos a probar el encendido con la nueva alimentación.

Problemas encontrados: El cable USB original dejó de funcionar y no permitía cargar el programa en el ESP32. Además, las pilas AAA de 1,2 V no daban la energía suficiente para que el módulo GPS encendiera.

Soluciones aplicadas: Se compró un cable USB nuevo para poder programar la placa. Para la alimentación, se reemplazaron las pilas AAA por pilas AA de 1,5 V y el profesor cambió el portapilas por uno de mayor capacidad, de modo que al conectarlas en serie sumaran el voltaje necesario.

Conclusión del día: Con el cable nuevo y la alimentación corregida, el ESP32 y el módulo GPS encendieron correctamente. Estos dos problemas quedaron resueltos y el prototipo pudo seguir usándose sin depender de la computadora.

Día 12 — 16/09/2026

Objetivo del día: Resolver el problema del envío de coordenadas y verificar que la página web recibiera la ubicación del GPS.

Actividades realizadas:

Revisamos de nuevo la conexión entre el ESP32 y el módulo GPS, prestando atención a los pines RX y TX.

Detectamos que el módulo NEO-6M venía configurado de fábrica a 9600 baudios, mientras que el programa lo leía a 115200, por lo que no se entendían.

Agregamos al programa el envío del comando $PUBX,41,1,0007,0003,115200,0*1E al iniciar, para cambiar la velocidad del módulo, y luego reiniciamos el puerto Serial2 a 115200.

Probamos el envío de las coordenadas al servidor y verificamos que la página web las mostrara en el mapa.

Problemas encontrados: El ESP32 y el módulo GPS no se comunicaban porque cada uno trabajaba a una velocidad distinta, y por eso la página web seguía sin recibir las coordenadas.

Soluciones aplicadas: Unificamos la velocidad de comunicación en 115200 baudios: el programa configura el módulo al encender y después abre el puerto Serial2 a esa misma velocidad. Con eso el ESP32 volvió a leer latitud, longitud y cantidad de satélites.

Conclusión del día: La página web recibió y mostró correctamente la ubicación enviada por el ESP32, lo que confirmó que el sistema completo —GPS, ESP32, servidor y página— funciona de forma integrada.

Día 13 — 23/09/2026

Objetivo del día: Realizar la prueba de lectura del GPS al aire libre y comprobar la precisión de las coordenadas obtenidas.

Actividades realizadas:

Llevamos el prototipo al patio del Colegio, en un lugar despejado y con vista libre del cielo.

Encendimos el dispositivo con las pilas AA y esperamos a que el módulo GPS detectara satélites.

Observamos en el monitor serie del Arduino IDE la cantidad de satélites y los valores de latitud y longitud.

Anotamos el tiempo que tardó en obtener la primera señal y comparamos las coordenadas con la ubicación real del colegio.

Registramos los resultados en la bitácora.

Problemas encontrados: El módulo tardó aproximadamente 30 minutos en obtener señal, mucho más de lo esperado. Además, solo detectó 6 satélites, porque los edificios y el techo del colegio reducían la cantidad de señales que llegaban al módulo.

Soluciones aplicadas: Esperamos con paciencia y mantuvimos el dispositivo en un lugar abierto, lejos de paredes y techos. Comprobamos que, aunque la primera conexión tarda más, el módulo conserva los datos de los satélites gracias a su batería de respaldo, por lo que los encendidos siguientes son más rápidos.

Conclusión del día: El módulo logró obtener coordenadas que coincidían con la ubicación real del colegio, lo que confirmó que la conexión con el ESP32 y la lectura de los datos funcionan correctamente. El tiempo de espera fue prolongado, pero se explica por ser la primera conexión y por los obstáculos del entorno.

Día 14 — 30/09/2026
Objetivo del día: Incorporar el buzzer al circuito y comprobar su funcionamiento como alerta sonora.

Actividades realizadas:

Revisamos el buzzer activo reciclado de proyectos anteriores del colegio y verificamos que funcionara.
Lo conectamos al ESP32 en los pines D23 y D22, según lo definido en la programación del equipo.

Cargamos el programa y comprobamos que al encender el dispositivo emitiera un pitido breve, como confirmación de que el sistema está activo.
Definimos un punto central (la casa) y un radio de zona permitida, y probamos que el buzzer emitiera una serie de pitidos al simular que la mascota salía de ese radio.
Ajustamos el volumen y la duración de los pitidos para que se escucharan sin ser molestos para el animal.

Problemas encontrados: Al principio el buzzer no sonaba porque estaba conectado con la polaridad invertida. Además, los pitidos eran demasiado seguidos y podían resultar molestos para la mascota.

Soluciones aplicadas: Corregimos la polaridad de las patas del buzzer y modificamos el programa para que los pitidos se repitieran con una pausa entre cada uno, de modo que la alerta sea clara pero no constante.

Conclusión del día:El buzzer quedó integrado al sistema y funciona correctamente: hace un pitido de confirmación al encender y avisa con una serie de pitidos cuando la mascota sale de la zona permitida. Con esto se completaron las pruebas del prototipo y dimos por cerrada la etapa de desarrollo del proyecto.

7.5. Manual de uso del dispositivo

Encendido

Verificar que las pilas estén colocadas con la polaridad correcta.

Accionar el interruptor hacia la posición de encendido.

El dispositivo emite un pitido breve, que confirma que el sistema funciona.

Esperar unos minutos hasta que el módulo GPS enganche señal de satélites (la primera vez puede tardar más).

Visualización de la ubicación

Abrir la página web del proyecto desde cualquier celular o computadora con internet.

El mapa muestra la última ubicación recibida del dispositivo.

La página se actualiza automáticamente cada vez que el ESP32 envía nuevas coordenadas.

Alerta de zona

Si la mascota sale del radio de zona permitida definido en el código, el buzzer emite una serie de pitidos para avisar al dueño.

Apagado

Accionar el interruptor hacia la posición de apagado.

No mover los cables con el dispositivo encendido.

Precauciones

Usar una sola fuente de energía a la vez (USB o pilas, nunca ambas).

Verificar la polaridad de las pilas antes de encender.

No exponer el dispositivo al agua.

8. Resultados y análisis

8.1. Resultados de la lectura del GPS

La prueba de lectura se realizó al aire libre, en el patio del Colegio Nacional E.M.D. Asunción Escalada, con el dispositivo alimentado por pilas AA.

Análisis: el módulo logró obtener coordenadas que coincidían con la ubicación real del colegio, lo que confirma que la conexión entre el GPS y el ESP32 y la lectura de los datos funcionaron correctamente. El tiempo de espera fue prolongado, pero se explica por dos factores: era la primera conexión del módulo en ese lugar y la señal se ve afectada por edificios, techos y otros obstáculos que reducen la cantidad de satélites visibles. Con 6 satélites se supera el mínimo de 4 necesario para calcular la posición, por lo que la lectura es válida.

8.2. Resultados del envío de datos y visualización

Una vez confirmada la lectura del GPS, se probó el envío de las coordenadas al servidor. El ESP32 se conectó al hotspot de un celular y comenzó a enviar los datos por el método POST cada 5 segundos. La página web recibió y mostró correctamente la ubicación en el mapa.

Análisis: el sistema completo —GPS, ESP32, servidor en Render y página web— funciona de forma integrada. La página muestra la última ubicación recibida y se actualiza cada vez que el ESP32 envía nuevas coordenadas, lo que permite conocer el punto donde se encuentra la mascota en ese momento, con un margen de error temporal de hasta 5 segundos.

8.3. Resultados del buzzer

El buzzer activo quedó integrado al sistema y responde de dos maneras: emite un pitido breve al encender, como confirmación de que el dispositivo está activo, y una serie de pitidos con pausa cuando la mascota sale del radio de zona permitida.

Análisis: la alerta sonora cumple su función de aviso inmediato para quien esté cerca del animal, sin depender de que el dueño esté mirando la página web en ese momento.

8.4. Resultados de la alimentación

El dispositivo funcionó correctamente con dos pilas AA de 1,5 V conectadas en serie, alojadas en el portapilas de mayor capacidad que aportó el profesor. Con las pilas AAA recargables de 1,2 V no fue posible encender el módulo GPS.

Análisis: el voltaje entregado por las pilas AAA resultó insuficiente para alimentar el conjunto ESP32-GPS. Las pilas AA, al tener mayor voltaje unitario, cubren el requerimiento del módulo. El dispositivo funciona sin estar conectado a una computadora, lo que permite colocarlo en el collar de la mascota.

8.5. Resultados económicos

La rifa organizada por el equipo alcanzó la meta prevista: se recaudó 1.000.000 de guaraníes, con 20 números por integrante a Gs. 10.000 cada uno. El costo total del prototipo fue de Gs. 267.000, por lo que quedó un remanente de Gs. 733.000. Los materiales reutilizados (cables, buzzer) y los aportados por el profesor (portapilas, interruptor, pilas AAA) no representaron gasto para el equipo.
Análisis: el costo final del prototipo es considerablemente menor al de un collar rastreador comercial, que además suele requerir una suscripción mensual, lo que confirma la viabilidad económica de la propuesta para una familia promedio. El remanente de la rifa aún no tiene un destino definido por el equipo.

9. Conclusiones y recomendaciones

9.1. Conclusiones

Sobre el objetivo general. El equipo logró diseñar y construir un módulo rastreador con GPS para mascotas de bajo costo, que obtiene la ubicación del animal y la envía por internet a un servidor propio, donde puede visualizarse en un mapa desde un celular o una computadora. El prototipo funciona de forma integrada: el GPS capta las coordenadas, el ESP32 las procesa y las transmite, el servidor las almacena y la página web las muestra.

Sobre los componentes. La combinación de un ESP32 DevKit V1 con un módulo GPS NEO-6M resultó adecuada para el proyecto: son económicos, fáciles de conseguir en el mercado local y cuentan con suficiente documentación para estudiantes que trabajan por primera vez con ellos. La comunicación por UART a través del puerto Serial2 funcionó correctamente una vez unificada la velocidad en 115200 baudios.

Sobre la programación. El desarrollo del programa exigió más tiempo del previsto, porque fue necesario reescribir el código desde cero y resolver la diferencia de velocidad entre el módulo GPS, configurado de fábrica a 9600 baudios, y el ESP32. Una vez corregido ese punto, la lectura de latitud, longitud y cantidad de satélites se volvió estable.

Sobre las pruebas. En la prueba al aire libre el módulo obtuvo coordenadas que coincidían con la ubicación real del colegio, con 6 satélites detectados y un tiempo de espera de aproximadamente 30 minutos en la primera conexión. El sistema completo —GPS, ESP32, servidor y página web— funcionó de forma integrada, y la página mostró la última ubicación recibida actualizándose cada 5 segundos.

Sobre los costos. El prototipo tuvo un costo total de Gs. 267.000, financiado con la rifa organizada por el equipo, que recaudó Gs. 1.000.000. Este monto es considerablemente menor al de un collar rastreador comercial, que además suele requerir una suscripción mensual, lo que confirma la viabilidad económica de la propuesta.

Sobre el aprendizaje del equipo. El proyecto permitió aplicar de forma práctica contenidos de electrónica, programación, redes y desarrollo web: soldadura de componentes, comunicación serial, peticiones HTTP, despliegue de un servidor en la nube y diseño 3D de una carcasa. También dejó enseñanzas sobre el trabajo en equipo y la resolución de problemas técnicos, ya que la mayoría de los avances surgieron después de varios intentos fallidos.

Sobre las limitaciones. El prototipo depende de la conexión a internet mediante el hotspot de un celular, por lo que no funciona en zonas sin cobertura. La primera conexión del GPS tarda varios minutos y la precisión puede verse afectada por edificios y techos. Además, la página muestra solo la última ubicación recibida, sin guardar el recorrido del animal.

9.2. **Recomendaciones y trabajos futuros**

Incorporar comunicación GSM. Agregar un módulo GSM con tarjeta SIM permitiría que el dispositivo envíe las coordenadas sin depender del hotspot de un celular, lo que ampliaría su uso

fuera del alcance de una red Wi-Fi.
Utilizar una batería recargable de mayor capacidad. Las pilas AA cumplen su función, pero una batería de litio recargable daría mayor autonomía y evitaría el gasto continuo en pilas.
Guardar el historial de ubicaciones. Modificar el servidor para almacenar las coordenadas anteriores permitiría mostrar el recorrido de la mascota en el mapa, y no solo su última posición.
Mejorar la carcasa. Se podría ajustar el diseño 3D para que sea más compacto y resistente al agua, de modo que el dispositivo soporte el uso diario en el collar.
Ampliar las pruebas. Conviene probar el dispositivo en distintos barrios de Asunción y en diferentes condiciones climáticas, para evaluar la precisión del GPS y el alcance de la señal en entornos más exigentes.
Documentar el código. Agregar comentarios al programa del ESP32 y al servidor facilitaría que otros estudiantes puedan reutilizarlo o continuar el proyecto en años siguientes.

10. Bibliografía

CircuitDigest. (2025, 15 de diciembre). Build a GPS tracker with Seeed Studio XIAO ESP32-S3. Hackster.io. https://www.hackster.io/CircuitDigest/build-a-gps-tracker-with-seeed-studio-xiao-esp32-s3-b66d05

Espressif Systems. (s. f.). ESP32 series datasheet. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Gupta, V. (2025, 31 de julio). IntelliCollar: A smart collar for livestock monitoring. Elecrow. https://www.elecrow.com/sharepj/intellicollar-a-smart-collar-for-livestock-monitoring-927.html

Hart, M. (s. f.). TinyGPSPlus [Software]. GitHub. https://github.com/mikalhart/TinyGPSPlus

Mohd Zahari, A. S., & Baharuddin, E. (2023). Outdoor pet tracking and health monitoring system. Evolution in Electrical and Electronic Engineering, 4(1), 72-78. https://doi.org/10.30880/eeee.2023.04.01.009

Prasojo, D. D., Ayuni, S. D., Anshory, I., & Wisaksono, A. (2026). Pet tracking system using Telegram notification. Journal of Electrical Engineering and Computer (JEECOM), 8(1). https://ejournal.unuja.ac.id/index.php/jeecom/article/download/12392/pdf

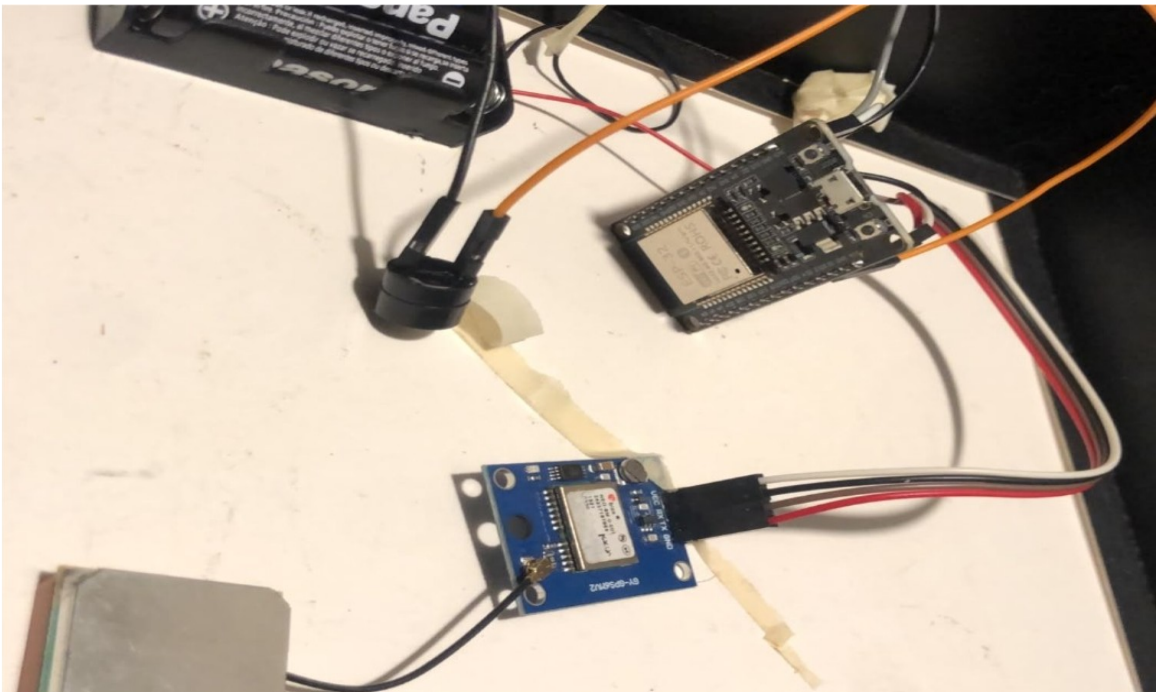
Render. (s. f.). Render documentation: Deploy web services. https://render.com/docs

u-blox. (s. f.). NEO-6 series: u-blox 6 GPS modules data sheet. https://www.u-blox.com

11. Anexos

Anexo A: Diagrama de conexión

Conexión de pines entre el módulo GPS NEO-6M (placa GY-GPS6MV2) y el ESP32 DevKit V1



El buzzer activo se conectó a los pines D23 y D22 del ESP32. Las pilas AA se conectan al portapilas y, a través del interruptor, al pin VIN y a GND

Anexo B: Código del ESP32 (Arduino IDE)

```
/*
  RASTREADOR GPS ESP32 + GY-GPS6MV2 -> RENDER + ALARMA DE DISTANCIA
  GPS TX -> D18 | GPS RX -> D5 | GPS VCC -> VIN | GPS GND -> GND
  Buzzer + -> D23 | Buzzer - -> D22 (hace de GND)
*/

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <TinyGPSPlus.h>
```

```
// ====== CONFIGURÁ ESTO ======
const char* WIFI_SSID  = "*****
const char* WIFI_PASS  = "*****
const char* SERVER_URL = "https://servicio-de-gps.onrender.com/api/update";
const char* API_KEY    = "*****;

const double RADIO_M = 20.0;      // distancia de la alarma (metros)
#define USAR_D22_COMO_GND true    // la pata - del buzzer va a D22
// ============================

#define RX_PIN 18
#define TX_PIN 5
#define GPS_BAUD 9600
#define BUZZER 23
#define BOTON_CASA 0              // botón BOOT de la placa
const unsigned long INTERVALO = 5000;

TinyGPSPlus gps;
HardwareSerial SerialGPS(2);
unsigned long ultimoEnvio = 0;

bool casaDefinida = false;
double casaLat = 0, casaLng = 0;
double distCasa = -1;
int fuera = 0;
volatile bool alarma = false;
volatile int bips = 0;

// El buzzer corre en una tarea aparte: suena aunque el envío por WiFi esté ocupado
void tareaBuzzer(void* p) {
  for (;;) {
    if (bips > 0) {
```

```
      digitalWrite(BUZZER, HIGH); vTaskDelay(120 / portTICK_PERIOD_MS);
      digitalWrite(BUZZER, LOW);  vTaskDelay(120 / portTICK_PERIOD_MS);
      bips--;
    } else if (alarma) {
      digitalWrite(BUZZER, HIGH); vTaskDelay(300 / portTICK_PERIOD_MS);
      digitalWrite(BUZZER, LOW);  vTaskDelay(700 / portTICK_PERIOD_MS);
    } else {
      digitalWrite(BUZZER, LOW);  vTaskDelay(50 / portTICK_PERIOD_MS);
    }
  }
}

bool buenaSenal() {
  return gps.location.isValid() && gps.location.age() < 5000 &&
         gps.hdop.isValid() && gps.hdop.hdop() < 4.0;
}

void definirCasa() {
  casaLat = gps.location.lat();
  casaLng = gps.location.lng();
  casaDefinida = true;
  alarma = false;
  fuera = 0;
  bips = 2;   // dos pitidos: casa guardada
  Serial.printf(">>> CASA DEFINIDA: %.6f, %.6f\n", casaLat, casaLng);
}

// Mantené apretado BOOT 1,5 segundos para guardar la casa en el lugar actual
void revisarBoton() {
  static unsigned long inicio = 0;
  static bool hecho = false;
  if (digitalRead(BOTON_CASA) == LOW) {
```

```
    if (inicio == 0) inicio = millis();
    if (!hecho && millis() - inicio > 1500 && buenaSenal()) { definirCasa(); hecho = true; }
  } else {
    inicio = 0;
    hecho = false;
  }
}
void revisarDistancia() {
  if (!gps.location.isUpdated() || !buenaSenal()) return;
  if (!casaDefinida) { definirCasa(); return; }   // la primera posición buena es la casa
  distCasa = TinyGPSPlus::distanceBetween(gps.location.lat(), gps.location.lng(), casaLat,
casaLng);
  if (distCasa > RADIO_M) {
    if (++fuera >= 3) alarma = true;              // 3 lecturas seguidas fuera
  } else {
    fuera = 0;
    if (distCasa < RADIO_M - 5) alarma = false;   // se apaga al volver a menos de 15 m
  }
}

void conectarWiFi() {
  if (WiFi.status() == WL_CONNECTED) return;
  Serial.print("Conectando WiFi");
  WiFi.disconnect(true);
  delay(200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  for (int i = 0; i < 40 && WiFi.status() != WL_CONNECTED; i++) {
    delay(500);
    Serial.print(".");
  }
  Serial.println(WiFi.status() == WL_CONNECTED ? " OK" : " FALLO");
```

```
}
int enviar(bool fix) {
  if (WiFi.status() != WL_CONNECTED) return -2;
  char json[200];
  if (fix) {
    snprintf(json, sizeof(json),
      "{\"fix\":true,\"lat\":%.6f,\"lng\":%.6f,\"speed\":%.1f,\"alt\":%.1f,\"sats\":%d,\"hdop\":%.1f}",
      gps.location.lat(), gps.location.lng(),
      gps.speed.isValid() ? gps.speed.kmph() : 0.0,
      gps.altitude.isValid() ? gps.altitude.meters() : 0.0,
      (int)gps.satellites.value(),
      gps.hdop.isValid() ? gps.hdop.hdop() : 99.0);
  } else {
    snprintf(json, sizeof(json), "{\"fix\":false,\"sats\":%d}",
      (int)gps.satellites.value());
  }

  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient http;
  http.setTimeout(20000);   // Render gratis puede tardar en despertar
  int code = -1;
  if (http.begin(client, SERVER_URL)) {
    http.addHeader("Content-Type", "application/json");
    http.addHeader("x-api-key", API_KEY);
    code = http.POST(json);
    http.end();
  }
  return code;
}

void setup() {
```

```
  Serial.begin(115200);

  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);
  pinMode(BOTON_CASA, INPUT_PULLUP);
#if USAR_D22_COMO_GND
  pinMode(22, OUTPUT);
  digitalWrite(22, LOW);    // D22 hace de GND para el buzzer
#endif

  SerialGPS.setRxBufferSize(1024);
  SerialGPS.begin(GPS_BAUD, SERIAL_8N1, RX_PIN, TX_PIN);
  xTaskCreate(tareaBuzzer, "buzzer", 2048, NULL, 1, NULL);
  bips = 1;   // pitido corto de arranque
  conectarWiFi();
}

void loop() {
  while (SerialGPS.available() > 0) gps.encode(SerialGPS.read());
  revisarBoton();
  revisarDistancia();

  if (millis() - ultimoEnvio >= INTERVALO) {
    ultimoEnvio = millis();
    if (WiFi.status() != WL_CONNECTED) conectarWiFi();

    bool fix = gps.location.isValid() && gps.location.age() < 5000;
    int code = enviar(fix);

    if (fix) Serial.printf("GPS %d sat | %.6f, %.6f | casa: %.0f m | %s | envio: %d\n",
      (int)gps.satellites.value(), gps.location.lat(), gps.location.lng(),
      distCasa < 0 ? 0.0 : distCasa, alarma ? "ALARMA" : "ok", code);
```

```
    else Serial.printf("Sin fix | %d sat | datos GPS: %lu | envio: %d\n",
      (int)gps.satellites.value(), (unsigned long)gps.charsProcessed(), code);
  }
}
```

Anexo C: Captura del monitor serie

```
GPS 0 sat | -25.283161, -57.632199 | casa: 0 m | ok | envio: 200
GPS 4 sat | -25.283240, -57.632097 | casa: 0 m | ok | envio: 200
GPS 4 sat | -25.283170, -57.632075 | casa: 0 m | ok | envio: 200
GPS 4 sat | -25.283113, -57.632172 | casa: 0 m | ok | envio: 200
GPS 4 sat | -25.283063, -57.632284 | casa: 0 m | ok | envio: 200
GPS 4 sat | -25.282998, -57.632468 | casa: 0 m | ok | envio: 200
GPS 3 sat | -25.282967, -57.632686 | casa: 0 m | ok | envio: 200
```

Anexo D: Captura de la página web con el mapa

